

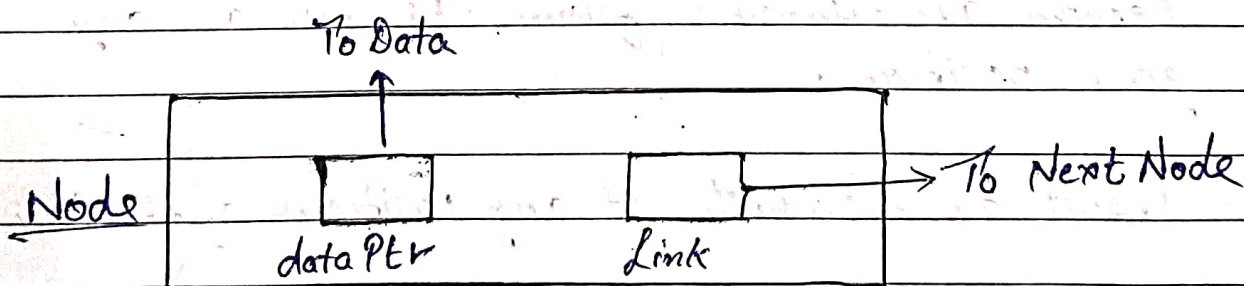
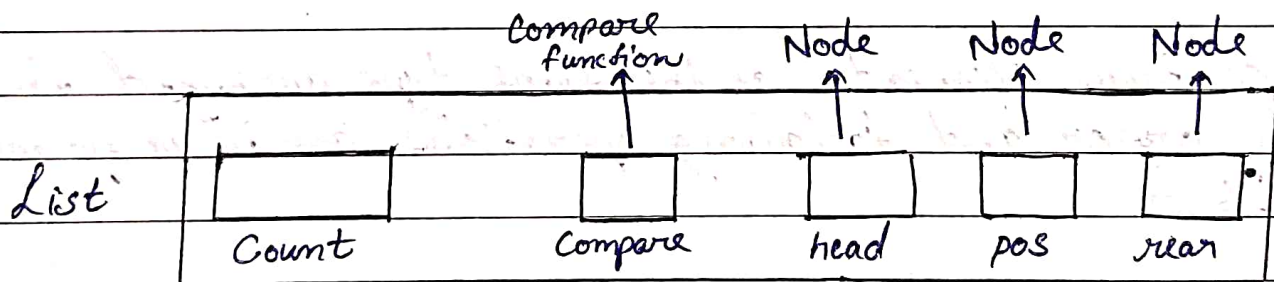
Unit - III List ADT (Abstract Data Type)

Abstract Data Type (ADT) is a type (or class) for objects whose behaviour is defined by a set of values and a set of operations.

There are three ADT's namely List ADT, Stack ADT, and Queue ADT.

• List ADT -

• The data is generally stored in a key sequence in a list that has a head structure consisting of count, pointers, and address of compare the function needed to compare the data in the list.



• The data node contains the pointer to a data structure and a self referential pointer that points to the next node in the list.

// List ADT Type Definitions -

```
typedef struct node
```

```
void *DataPtr;
```

```
struct node *Link;  
} Node;
```

```
typedef struct
```

```
{  
int count;
```

```
Node *pos;
```

```
Node *head;
```

```
Node *rear;
```

```
int (*compare)(void *argument1, void *argument2);  
} LIST;
```

A list contains elements of the same type arranged in sequential order and following operations can be performed on the list -

1) get() → Return an element from the list at any given position.

insert() → Insert an element at any position of the list.

remove() → Remove the first occurrence of any element from a non-empty list.

removeAt() → Remove the element at a specified location from a non-empty list.

-) replace() → Replace an element at any position with another element.
 -) size() → Return the number of elements in the list.
 -) isEmpty() → Return true if the list is empty, otherwise return false.
 -) isFull() → Return true if the list is full, otherwise return false.
- ~~~~~

- List is a common real world entity. List is nothing but a collection of objects of the same type. We can have a list of words, we can have a list of names, or we can have a list of numbers.
- So let us first define list as an abstract data type. So, when we define abstract data type, we just define the data that we'll store and then we define the operations available with the type and we do not go into the implementation details.

List (static list)

- Store a given number of elements of a given data-type.
- Write / modify element
- Read element at a position

* An array, gives us all these features *

Arrays functionalities -

```
int A[10];
```

```
A[1] = 2;
```

```
Print A[1];
```

• No, array is a data structure that gives us implementation for this list.

• No, now I want a list and that should have many more features. I want to handle it more scenarios for me. I will redefine this list here, I don't want a static list, a static collection of fixed size. I want dynamic list that should grow as per my need. -

List (dynamic list)

• Empty → if there are no elements in the list empty list has size 0.

• Insert → insert an element at any position in the list, in an existing list.

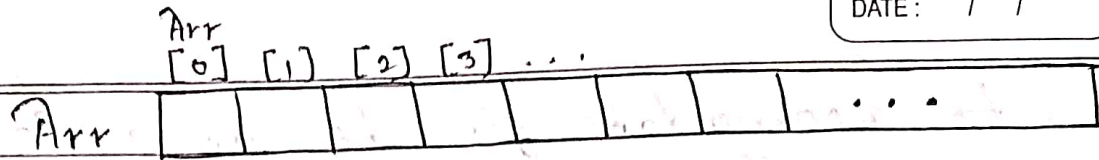
• Remove → remove element from the list.

• Count → Count the numbers of elements in the list.

• Read / modify element at a position.

• Specify the data type.

No, we can implement this particular list using Array.



```
int Arr [ MAXSIZE ];
```

```
int end = -1;
```

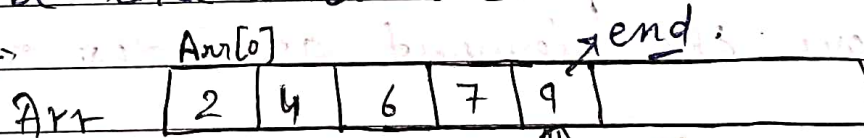
Here I'm defining a variable that will mark the end of the list in this array.

So, if the list is empty, we can initialize this variable or we can set this variable as minus one, because the lowest index possible is 0. So if end is minus one the list is empty at anytime a part of the array will store the list.

Here I'm trying to insert element (an integer) into this array.

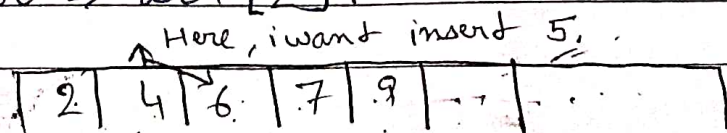
```
⇒ insert (2);
```

Let's say if we do not give the position at which the number is to be inserted, the number is always inserted towards the tail of the, towards the end of the list. So the list will be like we'll have an element at position zero.



And Now end is index zero, So at anytime end marks the this variable (9th in array (element)) end, marks the end of this list in this array.

Now, if I want to insert something in the list at a particular position, let say I want to insert number five at index two $\Rightarrow \text{arr}[2]$.



Then, to accommodate five here at $arr[2]$ particular position, we will have to shift all the elements one unit towards the right.

Before

2	4	6	7	9
---	---	---	---	---

After

2	4	<u>5</u>	6	7	9
---	---	----------	---	---	---

All the elements starting index two, we need to shift all the elements starting index two towards the right.

Code $\rightarrow$ insert (2)

insert (4)

insert (6)

insert (7)

insert (9)

insert (5, 2) (position, element)

- Now if i want to remove, an element from a particular position. I want to remove the element two so, i will pass the index zero here, i want to remove the element at index zero.

So, to do all the these elements after index zero will be shifted one unit.

$arr[0]$

Before - arr $\rightarrow$

2	4	6	7	9	9	...
---	---	---	---	---	---	-----

Remove

After.

4	6	7	9	.	.	...
---	---	---	---	---	---	-----

- Count - It will be equal to end plus one the value in the variable end plus one.

* Problem with Array.

⇒ We said that array will be of some large size, some max size.

But what is a good max size?

We can always exhaust the array, the list can grow to exhaust the array, there is no good max size.

• So we need to have a strategy for the scenario when the list will fill ~~to~~ the whole array.

• So, what do we do in that case?

We cannot extend the same array it is not possible to do so.

So, we will have to create a new array, a larger array. So, when the array is full, we will create a larger array, and copy all the elements from the previous array into the new array. and then we can free the memory for the previous array.

• Now the question is by how much should we increase the size of the new array, this whole operation of creating a new Array and copying all the elements from the previous array into the new array is costly.

So, let us see what are the costs in terms of time for all these operations, that we have in the dynamic list.

- The access to any element in the dynamic list, if we want to access it using index for read/write, then this will take constant time because,

We have an array here, and in array, elements are arranged in one contiguous block of memory, using the starting address or the base address of the block of the memory.

and the index or the position of the element can calculate the address of that particular element and excessive in constant time.

- Big Oh (O) notation, that is used to describe the time complexity of operations for constant time, it is written as in terms of big Oh,
 $\Rightarrow$ Constant time $\Rightarrow O(1)$

- If we wanted to insert element at the end of the array, end of the list i.e. then that again will be constant in time,

but if we would insert element at a particular position in the list, then we will have to shift element towards higher indices. In the worst case, we will have to shift all the elements to the right when we will be inserting at the first position, No time taken

for insertion, it will be proportional to the length of the list,

Let's say the length of the list is n
 $\rightarrow O(n)$

- Insertion $\rightarrow T \propto n$
 $O(n)$
- Removing $\rightarrow O(n)$
- Add (Adding an element at end of list) $\rightarrow O(n)$

List ADT Program

PAGE NO.:

DATE: / /

in Java

```
public class ListADT {  
    private static final int DEFAULT_CAPACITY = 10;  
    private int[] elements;  
    private int size;
```

```
    public ListADT() {  
        elements = new int [DEFAULT_CAPACITY];  
        size = 0;  
    }
```

```
    public void add (int value)  
    {  
        if (size == elements.length)  
        {  
            // If the array is full, double its capacity  
            int[] newElements = new int [2 * elements.length];  
            System.arraycopy (elements, 0, newElements, 0, size);  
            elements = newElements;  
        }  
        elements [size] = value;  
        size++;  
    }
```

```
    public int get (int index)  
    {  
        if (index < 0 || index >= size)  
        {  
            throw new IndexOutOfBoundsException(  
                "Index out of bounds");  
        }  
        return elements [index];  
    }
```

```
public boolean isEmpty()  
{  
    return size == 0;  
}
```

```
public int size()  
{  
    return size;  
}
```

```
public void display()  
{  
    System.out.print("[");  
    for (int i = 0; i < size; i++)  
    {  
        System.out.print(elements[i]);  
        if (i < size - 1)  
        {  
            System.out.print(", ");  
        }  
    }  
    System.out.println("]");  
}
```

// Driver Code

```
public static void main (String[] args)  
{  
    ListADT list = new ListADT();
```

```
list.add(1);  
list.add(2);  
list.add(3);
```

```
System.out.println("List size: " + list.size());  
System.out.println("Is the list empty? " +  
list.isEmpty());
```

```
System.out.println("List contents:");  
list.display();
```

```
int element = list.get(1);  
System.out.println("Element at index 1: " + element);
```

```
}
```

Output →

List size: 3

Is the list Empty? false

List contents: [1, 2, 3]

Element at index 1: 2

Stack ADT.

PAGE NO.:

DATE: / /

A stack is an abstract data type (ADT) that follows the Last-in-First-Out (LIFO) principle. We can implement a stack ADT in Java using arrays.
Code -

```
import java.util. EmptyStackException;
```

```
public class Stack {  
    private int maxSize;  
    private int top;  
    private int [] stackArray;
```

```
    public Stack (int size) {  
        this. maxSize = size;  
        this. stackArray = new int [maxSize];  
        this. top = -1;  
    }
```

```
    public void push (int value) {  
        if (isFull ()) {  
            System.out.println ("Stack is Full. Cannot push " + value);  
        } else {  
            stackArray [++top] = value;  
            System.out.println (value + " pushed to the Stack.");  
        }  
    }
```

```
    public int pop ()  
    {  
        if (isEmpty ()) {  
            throw new EmptyStackException ();  
        } else {
```

To be Continued...

```
        int poppedValue = stackArray[top--];  
        System.out.println(poppedValue + " popped from stack.");  
        return poppedValue;  
    }  
}
```

```
public int peek() {  
    if (isEmpty()) {  
        throw new EmptyStackException();  
    } else {  
        return stackArray[top];  
    }  
}
```

```
public boolean isEmpty() {  
    return top == -1;  
}
```

```
public boolean isFull() {  
    return top == maxSize - 1;  
}
```

// Driver Code

```
public static void main(String[] args) {  
    Stack stack = new Stack(5);  
    stack.push(1);  
    stack.push(2);  
    stack.push(3);  
    System.out.println("Top element: " + stack.peek());  
    stack.pop();  
    stack.pop();  
    stack.pop();  
}
```

To be Continued.

```
if (stack.isEmpty()) {
```

```
    System.out.println("Stack is empty.");
```

```
}
```

```
else {
```

```
    System.out.println("Stack is not empty");
```

```
}
```

```
}
```

```
}
```

Output -

1 pushed to the Stack

2 pushed to the Stack

3 pushed to the Stack

Top element: 3.

3 popped from the Stack.

2 popped from the Stack.

1 popped from the Stack.

Stack is Empty.

Queue ADT

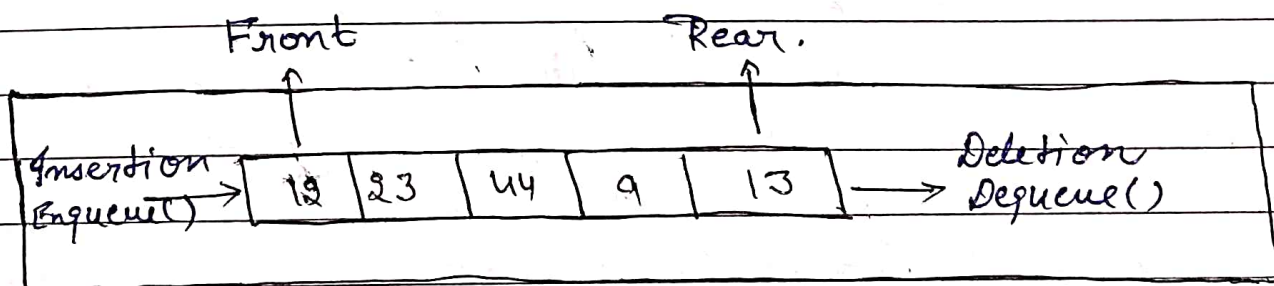
PAGE NO.:

DATE: / /

A Queue ADT (Abstract Data Type) in Data Structures and Algorithms is a linear data structure that follows the FIFO (First-In-First-Out) principle. It's similar to a real-world queue where the first person to arrive is the first one to be served.

In Queue ADT, elements are inserted at the rear (enqueue) and removed from the front (dequeue). The primary operations associated with a Queue ADT are:

- 1.) Enqueue → Adds an element to the rear of the queue.
- 2.) Dequeue → Removes an element from the front of the queue.
3. Front → Returns the element at front of the queue without removing it.
4. IsEmpty → Checks if the queue is empty.
5. Size → Returns the ~~number~~ number of elements in the queue.



Queue FIFO Operation

* Important points -

1. We have to import 'Queue' interface and use the 'LinkedList' class to implement a queue of integers.
2. We use 'offer()' to enqueue elements and 'poll()' to dequeue elements.
3. We use 'System.out.println(queue)' to display the content of the queue, which will show the elements in correct order.

code

```
import java.util. LinkedList;
import java.util. Queue;

public class Main
{
    public static void main (String[] args)
    {
        Queue<Integer> queue = new LinkedList<> ();
        // create a queue of integers

        queue.offer(1);
        queue.offer(2);
        queue.offer(3);
        System.out.println(queue); // [Output: [1,2,3]]

        int dequeuedItem = queue.poll();
        System.out.println("Dequeued: " + dequeuedItem);
        // Output: Dequeued 1.

        System.out.println(queue); // Output: [2,3]
```

* One more Example of Queue ADT

Being an interface the queue needs a concrete class for the declaration and the most common classes are the Priority Queue and LinkedList in Java.

• The Queue interface is declared as -

⇒ public interface Queue extends Collection

• Creating Queue Objects -

// Obj is the type of the Object to be stored in Queue.

Queue<Obj> queue = new PriorityQueue<Obj>();

#code -

```
import java.util. LinkedList;
```

```
import java.util. Queue;
```

```
public class QueueExample {
```

```
    public static void main(String[] args) {
```

```
        Queue<String> queue = new LinkedList<>();
```

```
        queue.add("Apple");
```

```
        queue.add("Banana");
```

```
        queue.add("Cat");
```

```
        System.out.println("Queue: " + queue);
```

```
        String front = queue.remove();
```

```
        System.out.println("Removed element: " + front);
```

```
        System.out.println("Queue after removal: " + queue);
```

```
        queue.add("date");
```

```
        String peeked = queue.peek();
```

```
        System.out.println("Peeked element: " + peeked);
```

```
        System.out.println("Queue after peek: " + queue);
```

```
    }
```

```
}
```

STACK

Intro -

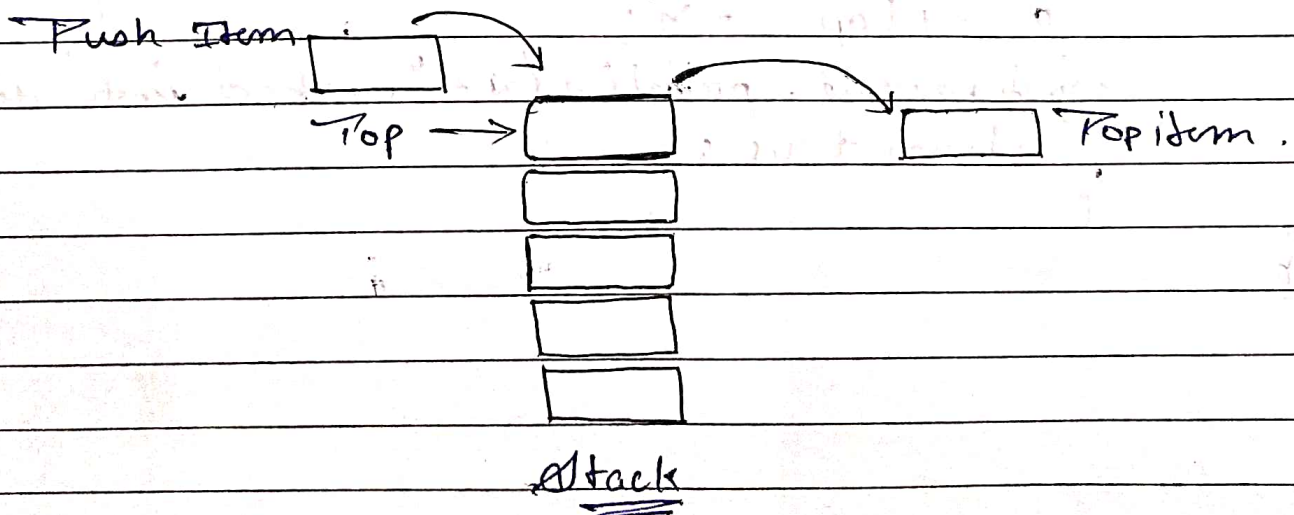
A stack is a linear data structure in which the insertion of a new element and removal of an existing element takes place at the same end and is represented as the top of the stack.

LIFO - (Last-In-First-Out) -

This strategy states that the element that is inserted last will come out first. You can take a pile of plates kept on top of each other as a real-life example. The plate which we put last is on the top and since we remove the plate that is at the top, we can say that the plate that was put last comes out first.

⇒ Basic Operations on Stack -

- `push()` → to insert an element into the stack.
- `pop()` → to remove an element from the stack.
- `peek/top()` → Returns the top element of the stack.
- `isEmpty()` → Returns true if stack is empty else false.
- `isFull()` → It checks whether the stack is full or not.
- `size()` → Returns the size of stack.



code -

class Stack {

static final int MAX = 1000;

int top;

int a[] = new int [MAX]; // Maximum size of stack

boolean isEmpty()

{

return (top < 0);

{

Stack()

{

top = -1;

{

boolean push(int x)

{

if (top >= (MAX - 1))

{

System.out.println("Stack Overflow");

return false;

{

else {

a[++top] = x;

System.out.println(x + " pushed into stack");

return true;

{

}

To be Continued...

```
int pop ()
{
    if (top < 0) {
        System.out.println ("Stack Underflow");
        return 0;
    }
    else {
        int x = a[top--];
        return x;
    }
}
```

```
int peek ()
{
    if (top < 0) {
        System.out.println ("Stack Underflow");
        return 0;
    }
    else {
        int x = a[top];
        return x;
    }
}
```

```
void print () {
    for (int i = top; i > -1; i--)
        System.out.print (" " + a[i]);
}
}
```

To be Continued...

// Driver Code

```
class Main
```

```
{  
    public static void main(String[] args)
```

```
{  
        Stack s = new Stack();
```

```
        s.push(10);
```

```
        s.push(20);
```

```
        s.push(30);
```

```
        System.out.println(s.pop() + "Popped from stack");
```

```
        System.out.println("Top element is: " + s.peek());
```

```
        System.out.print("Elements present in stack: ");
```

```
        s.print();
```

```
    }
```

```
}
```

Output -

- 10 pushed into stack
- 20 pushed into stack
- 30 pushed into stack
- 30 popped from stack
- Top element is : 20
- Elements present in stack : 20 10

Linked List

PAGE NO.:

DATE: / /

1. Link List - implementation using arrays & pointers

```
class Node {
```

```
    int data;
```

```
    int nextIndex; // Index of the next node
```

```
    Node (int data, int nextIndex)
```

```
    { this.data = data;
```

```
      this.nextIndex = nextIndex;
```

```
    }
```

```
}
```

```
class LinkedListExample {
```

```
    public static void main (String[] args)
```

```
    {
```

```
        int k = 5; // Number of pointers
```

```
        Node[] linkedList = new Node[10]; // Initialize  
                                           an array for linked list
```

```
        int currentIndex = 0; // Index of current node
```

```
        for (int i = 0; i < linkedList.length; i++)
```

```
        { linkedList[i] = new Node(-1, -1);
```

```
        }
```

```
// Insert elements at the beginning of the linked  
List
```

```
for (int i = 1; i <= 3; i++)
```

```
{ linkedList[currentIndex].data = i;
```

```
  linkedList[currentIndex].nextIndex = currentIndex + 1;
```

```
  currentIndex++;
```

```
}
```

// Traverse and print the Linked List

```
currentIndex = 0;  
while (currentIndex != -1)  
{  
    System.out.print (LinkedList [currentIndex]  
        .data + " -> ");
```

```
    currentIndex = LinkedList [currentIndex]  
        .nextIndex;  
}
```

```
System.out.println ("null");  
}
```

```
}
```

Output

1 → 2 → 3 → -1 → null

2. Singly link list program in Java without function

```
public class SinglyLinkedList {  
    static class Node  
        int data;
```

```
    Node(int data) {  
        this.data = data;  
        this.next = null;  
    }  
}
```

```
public static void main (String [] args)  
{
```

```
    Node head = new Node(1);  
    Node second = new Node(2);  
    Node third = new Node(3);
```

```
    head.next = second;  
    second.next = third;
```

// Display the elements in the linked list

```
    Node current = head;  
    while (current != null)  
    {
```

```
        System.out.print(current.data + " -> ");  
        current = current.next;
```

```
    }
```

```
    System.out.println("null");
```

// Insert a node at the end

```
Node newNode = new Node(4);  
third.next = newNode;
```

// Display the Updated List

```
current = head;  
while (current != null)  
{  
    System.out.println(current.data + " → ");  
    current = current.next;  
}  
System.out.println("null");
```

// Delete the second node (2)

```
head.next = third;
```

// Display the final list

```
current = head;  
while (current != null) {  
    System.out.print(current.data + " → ");  
    current = current.next;  
}  
System.out.println("null");  
}
```

Output — 1 → 2 → 3 → null
1 → 2 → 3 → 4 → null
1 → 3 → 4 → null

3. Doubly LinkedList ^{with} program in Java ~~function~~ functions.

```
public class DoublyLinkedList {  
    public static class Node {  
        int data;  
        Node next;  
        Node prev;  
  
        public Node (int data) {  
            this.data = data;  
            this.next = null;  
            this.prev = null;  
        }  
    }  
}
```

```
private Node head;  
private Node tail;
```

```
public DoublyLinkedList () {  
    head = null;  
    tail = null;  
}
```

```
public void insertAtFront (int data) {  
    Node newNode = new Node (data);  
    if (head == null) {  
        head = newNode;  
        tail = newNode;  
    } else {  
        newNode.next = head;  
        head.prev = newNode;  
        head = newNode;  
    }  
}
```

```
public void insertAtEnd (int data)
{
```

```
    Node newNode = new Node(data);
```

```
    if (tail == null) {
```

```
        head = newNode;
```

```
        tail = newNode;
```

```
    } else {
```

```
        newNode.prev = tail;
```

```
        tail.next = newNode;
```

```
        tail = newNode;
```

```
    } }
```

```
public void display () {
```

```
    Node current = head;
```

```
    while (current != null)
```

```
    {
```

```
        System.out.print (current.data + "<->");
```

```
        current = current.next;
```

```
    }
```

```
    System.out.println ("null");
```

```
}
```

```
public static void main (String [] args) {
```

```
    DoublyLinkedList doublyLinkedList = new DoublyLinkedList();
```

```
    doublyLinkedList.insertAtEnd (1);
```

```
    doublyLinkedList.insertAtEnd (2);
```

```
    doublyLinkedList.insertAtEnd (3);
```

```
doublyLinkedList.display (); // Output: 1<->2<->3<-> null
```

```
doublyLinkedList.insertAtFront (0);
```

```
doublyLinkedList.display (); // Output: 0<->1<->2<->3
```

```
<-> null
```

```
}
```

Queue

PAGE NO.:

DATE: / /

1 Single Queue program in Java (DSA):

```
import java.util.LinkedList;
import java.util.Queue;

public class SingleQueueProgram {
    public static void main (String[] args) {
        // Create a queue using the LinkedList class
        Queue<String> queue = new LinkedList<>();

        // Enqueue (add elements to the queue)
        queue.offer ("Element 1");
        queue.offer ("Element 2");
        queue.offer ("Element 3");

        // Dequeue (remove elements from the queue)
        String removedElement = queue.poll();
        System.out.println("Dequeued: " + removedElement);

        // Peek (view the front element without removing it)
        String frontElement = queue.peek();
        System.out.println("Front element: " + frontElement);

        // Print the entire queue
        System.out.println("Queue elements: " + queue);

        // Check if the queue is Empty
        boolean isEmpty = queue.isEmpty();
        System.out.println("Is the queue empty: " + isEmpty);
    }
}
```

2. Double-ended queue program in JAVA (DSA)

```
import java java.util.Deque;
import java.util.LinkedList;
```

```
public class DoubleQueueExample {
    public static void main (String [] args)
    {
```

```
        // Create a LinkedList to implement a deque
        Deque <String> deque = new LinkedList <>();
```

```
        // Add elements to the front of the deque
```

```
        deque.addFirst ("First");
        deque.addFirst ("Second");
        deque.addFirst ("Third");
```

```
        // Add elements to the rear of the deque
        deque.addLast ("Last");
        deque.addLast ("Penultimate");
```

```
        // Display the elements in the deque
        System.out.println("Deque elements: " + deque);
```

```
        // Remove elements from the front / rear of the deque
        String firstElements = deque.removeFirst();
        String lastElements = deque.removeLast();
```

// Display the removed elements

```
System.out.println("Removed first element: " + firstElement);
```

```
System.out.println("Removed last element: " + lastElement);
```

// Display the elements in the deque after removal

```
System.out.println("Updated deque elements: " + deque);
```

// Peek at the first and last elements without removing them.

```
String peekFirst = deque.peekFirst();
```

```
String peekLast = deque.peekLast();
```

```
System.out.println("First element (peek): " + peekFirst);
```

```
System.out.println("Last element (peek): " + peekLast);
```

```
}
```

```
}
```

Output →

Deque elements: [Third, Second, First, Last, Penultimate]

Removed First element: Third

Removed Last element: Penultimate

Updated deque elements: [Second, First, Last]

First element (peek): Second

Last element (peek): Last

3. Circular Queue program in JAVA (DSA)

```
public class CircularQueue {  
    private int[] array;  
    private int front;  
    private int rear;  
    private int size;  
  
    public CircularQueue (int capacity) {  
        array = new int [capacity];  
        front = -1;  
        rear = -1;  
        size = 0;  
    }  
  
    public boolean isEmpty () {  
        return size == 0;  
    }  
  
    public boolean isFull () {  
        return size == array.length;  
    }  
  
    public void enqueue (int item)  
    {  
        if (isFull ()) {  
            System.out.println ("Queue is Full. cannot  
                                enqueue.");  
        }  
        return;  
    }  
}
```

```
if (isEmpty()) {
```

```
    front = 0;
```

```
    rear = 0;
```

```
} else {
```

```
    rear = (rear + 1) % array.length;
```

```
}
```

```
array[rear] = item;
```

```
size++;
```

```
}
```

```
public int dequeue() {
```

```
    if (isEmpty()) {
```

```
        System.out.println("Queue is empty. Cannot dequeue.");
```

```
        return -1;
```

```
    }
```

```
    int item = array[front];
```

```
    if (front == rear) {
```

```
        front = -1;
```

```
        rear = -1;
```

```
    } else {
```

```
        front = (front + 1) % array.length;
```

```
    }
```

```
    size--;
```

```
    return item;
```

```
}
```

```
public int size() {
```

```
    return size;
```

```
}
```

```
public int peek() {  
    if (isEmpty()) {  
        System.out.println("Queue is empty. Cannot peek.");  
        return -1;  
    }  
    return array[front];  
}
```

```
public void display() {  
    if (isEmpty()) {  
        System.out.println("Queue is empty.");  
        return;  
    }  
    int i = front;  
    do {  
        System.out.print(array[i] + " ");  
        i = (i + 1) % array.length;  
    } while (i != (rear + 1) % array.length);  
    System.out.println();  
}
```

```
public static void main (String[] args)
```

```
{
```

```
    CircularQueue queue = new CircularQueue(5);
```

```
    queue.enqueue(1);
```

```
    queue.enqueue(2);
```

```
    queue.enqueue(3);
```

```
    queue.enqueue(4);
```

```
    queue.enqueue(5);
```

```
System.out.println("Dequeued element: " +  
queue.dequeue());
```

```
System.out.println("Peeked element: " +  
queue.peek());
```

```
System.out.println("Queue size: " + queue.size());
```

```
System.out.println("Enqueue 6");  
queue.enqueue(6);
```

```
System.out.println("Queue elements:");  
queue.display();
```

```
}
```

```
}
```

Output →

Queue elements: 1 2 3 4 5

Dequeue element: 1

Peeked element: 2

Queue size: 4

Enqueueing 6

Queue elements: 2 3 4 5 6

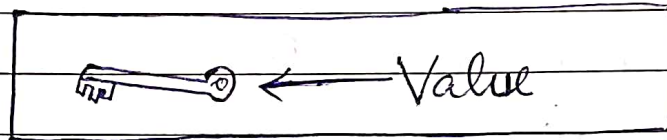
Dictionaries

1. Dictionaries are one of the most abstract data structure -

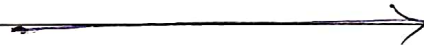
- They are also called Maps and Associative Arrays

• Dictionary -

An abstract Data Structure which stores data in the form of key/values pairs.



- Think of a key/value pair like a social security number.



value

Social Security
0000-0000

Aman

0000-00-0000

- We index dictionaries using these keys instead of a numerical index.

Array -

Index value →	Index	0	1	2	3	4	5	6
	Numbers	1	2	3	4	5	6	7

Dictionaries

Index	Key	Key	Key	Key	Key
Numbers	1	2	3	4	5

- The keys in a key / value pair can be pretty much any primitive data type you can think of.

Like → keys can be → Integers
Strings
Double.

Integers

Keys	0	1	2	3
Values				

Keys	0	1	2	3
Values	"Apple"	"Cat"	"Dog"	"Emly"

Strings

Keys	"Apple"	"Cat"	"Dog"	"Emly"
Values				

Keys	"Apple"	"Cat"	"Dog"	"Emly"
Values	true	false	true	true

Double

Keys	1.2	3.4	5.6	2.4
Values				

Keys	0.5	1.5	4.2	3.5
Values	Dictionary	Dictionary	Dictionary	Dictionary

- There are lot of flexibility as for the values well we have even more flexibility with these we can have keys in a dictionary correspond to pretty much anything strings, Booleans, even another dictionary with its own set of key value pairs.

- There are 2 extremely important restrictions when it comes to dictionaries -
 - Every key can only appear once within the dictionary.
 - Each key can only have one value.
- * Each Key has to be Unique *

- There can be duplicates of values within a dictionary.

↳

keys	0	1	2	3	4	5
Value	"Apple"	"Banana"	"Cat"	"Apple"	"Dog"	"Emly"

Duplicates values allowed.
because keys are different.

- Dictionaries - Time Complexity Equations.

Dictionaries are implemented with Hash Tables.

Index Keys	0	1	2	3	4	5	6
Values	0	0	2	0	4	0	6

No values are Nil
 Nil
 Nil

→ Why Doesn't this work?

- This doesn't work because it's based upon the simple assumption that the size of the array is not too large.

Sure this might work great

- for a dictionary where we have 10 or so on elements. but's lets say instead of 10 elements we have dictionary we want to use which has 10 values which are sporadically placed from 1 to a billion.

Index	0	10	100	1,000	100k	1M	...
Value	1	10	100	1,000	10,000	1 Million	...

We want keep the keys as being in the same index position as their values so that we know

exactly where they are and can run operations in constant time but my count that is from 0 to 1 billion.

9,999,990 Values

Taking up space in memory and that not good whatsoever,

So what's the answer $\rightarrow$ Hash Tables

Dictionaries - Hash Tables (Mini Lesson)

- Hash Tables are a way to store this information so that we're able to cut down the amount of nil values while also following for the information to be stored in a way that is easily accessible.

eg:

Index	0	1	2	3	4	5	6	...
Values	1	10	100	1000	10K	100K	1 Mill.	...

Using Hash Functions:

- A Hash Function will take all the keys for a given dictionary and strategically map them to a certain index location in an array so that it can eventually be retrieved easily.
- The goal of a good hashing function is to take in a key and reliably place it somewhere in the hash table so that it can be accessed later.

Dictionary contains keys which are integers from 1 to a billion by a factor of 10. So, the keys would be 1, 10, 100, 1000, 10k, 100k, 1 Million, 10 Million, 100 Million, 1 Billion.

Key	1	10	100	1000	10k	100k	1M	10M	100M	1B
Index	0	1	2	3	4	5	6			

↓
Good Hash Function for this —

* Take the key/number, divide it by itself, then multiply the result by the number of digits in the key minus 1.

Ques So find out where to put the key 1 Million. We take a million divided by itself which yields one and then multiply that by the number of digits in the integer minus 1.

$$(1,000,000 / 1,000,000) \times (7 - 1)$$

↓
1

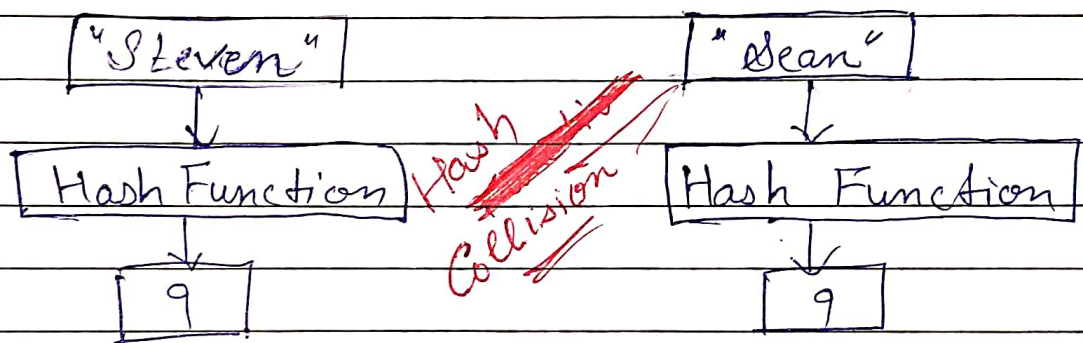
Output → 6

That means we store the key 1 Million at index 6. Index location

- Dictionaries are built upon these Hash Tables, and the key's in our key/value pairs are stored in memory IN these hash Tables at indexes which are determined by a hash function.

★ Ques - What happens when we run two different dictionary keys into a hash function, and they return the same index location to be stored within the hash table?

For eg.



1) Open Addressing - (Table)

⇒ Put the key in some other index location separate from the one returned to us by the hash function.

8	9	10
"selena"	nil	nil

2) Closed Addressing - Uses linked lists to chain together keys which result in the same hash value.

8	9	10
"selena"	"Steven"	nil
	"Sean"	